

## A Detailed Look at the Waveshare ESP32-C6 1.9-inch ST7789 TFT Display with Touch Interface

This tutorial covers the Waveshare ESP32-C6 development board featuring a 1.9-inch TFT display with a resolution of 170 x 320 pixels and an optional touch interface. Although the board is described on the product page (<https://www.waveshare.com/esp32-c6-lcd-1.9.htm>) and in the wiki (<https://docs.waveshare.com/ESP32-C6-LCD-1.9>), there are still a few surprises encountered during use—surprises I'd like to save you from experiencing. Naturally, all code examples are available in my GitHub repository, which might just end up becoming the "unofficial" product repository :-)



## Board Feature Overview

- ESP32-C6FH8 processor with 8 MB flash memory and 8 MB PSRAM

- 1.9-inch TFT display with ST7789V2 chip and 170 x 320 pixel resolution

- Optional: Touch interface with CST816 chip \*1)

- Optional: WS2812 RGB LED \*1)

- Optional: IPEX1 connector for an external antenna \*1)

- MicroSD card reader

- QMI8658 6-axis motion (IMU) sensor

- TCA9554 8-channel GPIO expander

- JST 1.25 connector for a 3.7V lithium battery, featuring integrated charge/discharge circuitry and a charging status LED

- Reset and Boot buttons

\*1) If you select the touch option, you also get the IPEX1 connector; conversely, if you choose the "without touch" option, the RGB LED is included instead.

## TFT Display

If you have already worked with the ST7789 display type, getting this display up and running will be quite easy. The SPI interface pins are documented, and for initial testing, I use the Adafruit ST7735/ST7789 graphics library.

However, there are two specific details to note: to achieve high speed (especially important for frequent drawing operations), you should use the hardware SPI mode, which I configure for the correct pins as follows:

```
SPI.begin(TFT_SCLK, TFT_MISO, TFT_MOSI, TFT_CS);
```

The second detail concerns the backlight circuitry: it is controlled via a MOSFET and operates inversely—meaning `digitalWrite(TFT_BL, LOW)` turns the backlight on. To ensure compatibility with other projects, I am providing programs for the three graphics libraries "Adafruit ST7789", "TFT\_eSPI", and "LovyanGFX". Please note that you need a patched library for TFT\_eSPI. You can find details on this in my tutorial "Getting Started with an ESP32-C6 Supermini device connected to an ST7789 TFT display".

## Touch Interface

Waveshare documents the pinout on the product page, and I attempted to get the CST816 driver chip working using several different libraries. This chip is controlled and queried via the I2C interface; additionally, two lines—Interrupt (IRQ) and Reset (RST)—are involved.

Unfortunately, none of the libraries worked or registered any touch events. I eventually found a clue in the official GitHub repository for the board within the "Arduino/08\_LVGL\_Test" example (specifically in the `getTouch`` function): although the IRQ and RST pins are listed and appear in the schematic, they are not physically connected to the processor. They are marked as "NC" (Not Connected) on the product page, a detail that is very easily overlooked. Why are these pins explicitly mentioned if they cannot actually be used?

This explains why none of the standard libraries worked, as they all require one or both of these pins to detect touch events.

The solution is to continuously read ("poll") a specific register of the CST816 chip via the I2C interface to detect touches and retrieve the screen coordinates. I have provided an example program for this as well, though only for the TFT\_eSPI graphics library (for the microSD card reader, I created an example program compatible with all three graphics libraries that also includes touch detection).

## MicroSD Card Reader

Since the ESP32-C6 has only one available SPI interface, four of the SD card reader's SPI pins are shared with the display's pins; only the "Chip Select" ("SD\_CS") pin is assigned its own dedicated pin. As with any shared bus system, the order in which the various devices (in this case, the TFT display and the SD card reader) are initialized is crucial. I am therefore providing an example program for the three graphics libraries that displays two JPG images and two BMP images on the screen. The next image appears each time the screen is touched. I use the "JPEGDEC" library for JPG images, whereas I draw BMP images pixel by pixel—a process that is significantly slower than rendering JPGs.

## QMI8658 Motion Sensor

This sensor is an IMU (Inertial Measurement Unit)—a spatial sensor system that measures an object's motion, orientation, and attitude in 3D space. It is used for precise orientation and navigation, often serving as a backup when GPS signals are unavailable. It measures acceleration and angular velocity for the X, Y, and Z axes, which is why it is also referred to as a 6-axis sensor. It communicates via the I2C interface, and I use the QMI8658 Arduino library to retrieve the sensor data. In addition to the six values mentioned, the chip includes a built-in temperature sensor, the readings of which can also be accessed.

I haven't tried the much more professional "SensorLib" library, though it is said to work with this sensor as well.

This sensor works reliably and without surprises; however, Waveshare failed to specify the orientation of the three axes. To ensure we are referring to the same directions, let's place the board flat on a table in landscape orientation with the USB port facing left:

- X-axis: points forward, away from the long edge
- Y-axis: points to the right, away from the short edge
- Z-axis: points upwards, away from the display

The directions mentioned yield positive (increasing) values, while the opposite directions yield negative (decreasing) values.

I will demonstrate how it works using the Adafruit graphics library.

## Battery Connection

You can connect a 3.7-volt lithium battery to the two-pin JST 1.25 connector here; it is charged via the USB-C port. The polarity follows the standard convention, but you should verify your board's polarity before connecting the battery to avoid destroying the battery or causing further damage.

A green LED next to the USB port lights up when the battery is charging.

Discharge protection: The board lacks discharge protection; therefore, you should only use batteries with a built-in Battery Management System (BMS). Otherwise, the battery could undergo uncontrolled deep discharge, which is detrimental to its lifespan.

However, the board does feature integrated voltage monitoring connected to GPIO 0 via a resistive voltage divider. Unfortunately, unlike on some other boards, this monitoring circuit cannot be switched on or off; consequently, even during long periods of inactivity (such as deep sleep), a small but constant current flows through the voltage divider, which will eventually drain the battery.

According to the datasheet, the voltage divider consists of 200k $\Omega$  and 100k $\Omega$  resistors, reducing the input voltage to one-third of its value: 3.7 volts becomes 1.233 volts at the Analog-to-Digital Converter (ADC) input. Since the readings fluctuate despite a constant voltage, I read 20 values and calculate the average to obtain a more stable reading.

I divide the resulting values by the constant ``BATTERY_VOLTAGE_DIVIDER`` (322.7). The "irregular" value arises from the calculated 1/3 divider (since we receive the raw values in millivolts, we divide by 333); however, every component has a tolerance, which in practice would result in a slightly inaccurate reading. I simply measured the actual voltage with a multimeter and then adjusted the divider so that the values displayed on the ESP32-C6 and the multimeter matched (this constitutes the "calibration").

## TCA9554 GPIO Extender

This board features a chip I haven't encountered on any of my previous boards: the TCA9554 8-channel GPIO extender. Since the SPI and I2C interfaces already utilize a significant number of GPIOs, this extender is employed to provide additional I/O capability. It uses the I2C interface, and with the help of the Adafruit xCA9554 library, you can utilize 8 extra GPIOs for input or output tasks. On the board's pinout, these are labeled EXIO0 through EXIO7. Using the library, you define whether each port should operate as an input or an output.

In my example, eight red buttons are displayed on the screen; touching a button changes its color to green and lights up the LED connected to the corresponding port, while a subsequent touch turns the LED off again (Adafruit graphics library).



## **Does this board have any downsides?**

At first glance, it looks like the perfect board; however, there is one weak point: the SPI interface. The ESP32-C6 has an SPI interface that is shared between the display and the microSD card reader (bus sharing). As you have seen, this parallel operation works excellently.

However, the manufacturer does not route the base SPI pins to the breadboard-compatible headers, meaning you cannot connect additional SPI devices (such as an NFC card reader). Of course, there are small adapter boards you could insert into the reader slot to access the signals, but that would be more of a temporary DIY solution than a permanent setup.

## **Official GitHub Repository**

Waveshare does provide a dedicated repository for this board that includes feature examples. In addition to Arduino IDE support, there are examples for ESP-IDF, as well as the pre-installed firmware available as a ready-to-upload binary. I found these examples unhelpful, however, as they are spread across multiple files and offer no guidance on controlling the display or the touch functionality.

Note: Since this board has only been on sale since March 2026, the examples are just three months old—perhaps the manufacturer will expand upon them in the future.

## Download links and version details of the software used

- Arduino IDE 2.3.8: <https://www.arduino.cc/en/software>
- esp32 Boards 3.3.8 (based on ESP-IDF v5.5.4):  
<https://github.com/espressif/arduino-esp32>
- Adafruit ST7735|ST7789 1.11.0:  
<https://github.com/adafruit/Adafruit-ST7735-Library>
- Adafruit GFX 1.12.6: <https://github.com/adafruit/Adafruit-GFX-Library>
- TFT\_eSPI 2.5.43: [https://github.com/Bodmer/TFT\\_eSPI](https://github.com/Bodmer/TFT_eSPI) (patched version)
- TFT\_eSPI (patched version) based on 2.5.43:  
[https://github.com/AndroidCrypto/TFT\\_eSPI](https://github.com/AndroidCrypto/TFT_eSPI)
- LovyanGFX 1.2.21: <https://github.com/lovyan03/LovyanGFX>
- CST816 Library (without version): created by myself
- SD SD-Card Reader 1.3.0: <https://github.com/arduino-libraries/SD> (is included in esp32 boards)
- JPEGDEC JPG Bibliothek 1.8.4:  
<https://github.com/bitbank2/JPEGDEC>
- QMI8658 Arduino Library 1.0.1:  
<https://github.com/lahavg/QMI8658-Arduino-Library>
- Adafruit xCA9554 1.0.0:  
[https://github.com/adafruit/Adafruit\\_XCA9554](https://github.com/adafruit/Adafruit_XCA9554)
- Waveshare Manufacturer Store Page:  
<https://www.waveshare.com/esp32-c6-lcd-1.9.htm>
- Waveshare Manufacturer GitHub Repository:  
<https://github.com/waveshareteam/ESP32-C6-LCD-1.9>

## App Source Code

You can find all the code collected in my GitHub repository:  
<https://github.com/AndroidCrypto/ESP32-C6-LCD-1.9-inofficial>. For the patched TFT\_eSPI library, please refer to my tutorial titled "Getting Started with an ESP32-C6 Supermini device connected to an ST7789 TFT display," published here:  
<https://medium.com/@androidcrypto/getting-started-with-an-esp32-c6-supermini-device-connected-to-an-st7789-tft-display-76b0809e292f>